

Binary Search

Kanishk Goel

10 May, 2026

1 Binary Search

Binary Search
<pre>left $\leftarrow$ 0 $n \leftarrow$ length of array A right $\leftarrow n - 1$ while $left \leq right$ do mid $\leftarrow \frac{left+right}{2}$ if $mid = target$ then return mid else if $mid \leq target$ then left $\leftarrow mid+1$ else right $\leftarrow mid-1$ return <i>Not Found</i></pre>

2 Lower-Upper Bound

Lower Bound
<pre> left ← 0 first_match ← -1 n ← length of array A right ← n - 1 while left ≤ right do mid ← $\frac{\text{left} + \text{right}}{2}$ if mid = target then right = mid-1 first_match = mid else if mid ≤ target then left ← mid+1 else right ← mid-1 return first_match </pre>

For upperbound, the $\text{right} = \text{mid} - 1$ condition in the if block will change to $\text{left} = \text{mid} + 1$, rest remains the same.

3 Binary Search for Answer

Problem: All numbers are either good or bad. if x is good, so is $x + 1$. Let f be a function that determines if x is good or not i.e

$$f(x) = \begin{cases} 1, & x \text{ is a good number,} \\ 0, & \text{otherwise} \end{cases}$$

finally, we use this f to determine if x is good or not during the binary search. The binary search runs until $\text{left} + 1 = \text{right}$.

Runtime: $O(T(f) \cdot \log(r - l))$ where $T(f)$ is the time complexity of running f .

3.1 Continuous Search

When $f : \mathbb{R} \rightarrow \mathbb{R}$ be a real-valued function that is continuous on the segment $[L, R]$.

Problem: We have n ropes, with a_i lengths respectively. We want to cut k pieces in total from all the ropes. Determine maximum length of the piece we can cut.

Idea:

$$f(x) = \sum_{i=1}^n \lfloor \frac{a_i}{x} \rfloor$$

determines if the value x works, we determine some ϵ as the max precision we can reach. So, we need at least $\log(\frac{\text{maxans}}{\epsilon})$ number of iterations from l to r with ϵ step size. Therefore, we use a for loop for these many iterations.

Runtime: $O(n \cdot \log \frac{1}{\epsilon})$

Note: We cannot use while loop, because double only stores values till a certain point, and if the answer is adjacent to l and r , we could get stuck in a infinite loop, as halving l and r will never yield the answer we are looking for (occurs when the precision required is high enough).

4 Minimax Problems

Problems of the format: $\min(\max(\dots))$

4.1 Problem 1

Consider a number line on which people live. For each person i , we know the coordinate x_i and their walking speed v_i . They are all to walk towards a point x . We have to choose x such that, the time required for all of them to assemble at x is the least.

Solution: Time required for a person to reach x is equal to:

$$T_i(x) = \frac{|x_i - x|}{v_i}$$

thereby total time, $T_{\text{total}} = \max_i T_i(x)$.

We binary search over possible times t , to find $T_{\text{total}} \leq t$. This reduces the problem to finding if $\forall i, \bigcap_{i=1}^n [x_i - v_i \cdot t, x_i + v_i \cdot t]$ is a non-empty segment.